

Update Job Schema & Migration

So we have very basic create and read functionality for jobs. The next step is to update the job schema and migration to include the rest of the fields that we need for a job listing. We will also update the form to include these fields. A schema is just a definition of the table and the fields that it contains. A migration is a way to update the schema of a table.

Create a New Migration

When we want to change the schema of a table we need to create a new migration. Technically you could do a rollback and then use the same migration file, but usually that would be a bad idea because it would be hard to track down the changes that were made. Any changes that you make should be in a new migration file.

An issue that we could run into when we run a migration that adds fields to the table is getting an error because there is already data in the table. We have a bunch of options here. You could manually delete the records first, but another approach is to clear the data in the migration file. This is what we will do.

We need to run the following command to create a new migration:

```
php artisan make:migration add_fields_to_job_listings_table --table=job_listings
```

We need to specify the table name because we didn't use the regular convention of using the plural form of the model name because there was already a table named `jobs` in the database. We used the `--table` option to specify the table name.

Here is an example of a job listing and the fields that we need:

```
[
  "id" => 1,
  "user_id" => 1,
  "title" => "Software Engineer",
  "description" => "As a Software Engineer at Algorix, you will be responsible
for designing, developing, and maintaining high-quality software applications. You
will work closely with cross-functional teams to deliver scalable and efficient
solutions that meet business needs. The role involves writing clean, maintainable
code, participating in code reviews, and staying current with industry trends to
ensure our technology stack remains cutting-edge.",
  "salary" => 90000,
  "tags" => "development, coding, java, python",
  "job_type" => "Full-time",
  "remote" => false,
  "requirements" => "Bachelors degree in Computer Science or related field, 3+
years of software development experience",
  "benefits" => "Healthcare, 401(k) matching, flexible work hours",
  "address" => "123 Main St",
  "city" => "Albany",
```

```

    "state" => "NY",
    "zipcode" => "12201",
    "contact_email" => "info@algorix.com",
    "contact_phone" => "348-334-3949",
    "company_name" => "Algorix",
    "company_description" => "Algorix is a leading tech firm specializing in
innovative software solutions and cutting-edge technology.",
    "company_logo" => "logos/logo-algorix.png",
    "company_website" => "https://algorix.com"
];

```

up Method

Open the new migration file and add the following code to the `up` method to add the new fields to the table:

```

public function up(): void
{
    // Clear the table
    DB::table('job_listings')->truncate();

    // Modify the table schema
    Schema::table('job_listings', function (Blueprint $table) {
        $table->integer('salary');
        $table->string('tags')->nullable();
        $table->enum('job_type', ['Full-Time', 'Part-Time', 'Contract', 'Temporary',
'Internship', 'Volunteer', 'On-Call'])->default('Full-Time');
        $table->boolean('remote')->default(false);
        $table->text('requirements')->nullable();
        $table->text('benefits')->nullable();
        $table->string('address')->nullable();
        $table->string('city');
        $table->string('state');
        $table->string('zipcode')->nullable();
        $table->string('contact_email');
        $table->string('contact_phone')->nullable();
        $table->string('company_name');
        $table->text('company_description')->nullable();
        $table->string('company_logo')->nullable();
        $table->string('company_website')->nullable();
    });
}

```

We are first truncating the table to clear the data. This is not something that you would normally do in a migration, but we are doing it here because we are adding new fields to the table and we don't want to get an error because there is already data in the table. Truncate will not only delete the records, but it will also reset the auto-incrementing id.

We are adding a bunch of new fields. It is all pretty self-explanatory. For job type, we are using an enum to limit the values to a set of predefined values and using a default of `Full-Time`. For remote, we are using a

boolean to indicate if the job is remote or not. For the rest of the fields, we are using the regular string and text types. The tags will be a comma-separated list of tags. Anything labeled as **nullable** can be left blank.

down Method

For the down, we will just drop all of the columns we added in the **up** method. Add the following code to the **down** method:

```
public function down(): void
{
    Schema::table('job_listings', function (Blueprint $table) {
        $table->dropColumn(['salary', 'tags', 'job_type', 'remote',
        'requirements', 'benefits', 'address', 'city', 'state', 'zipcode',
        'contact_email', 'contact_phone', 'company_name', 'company_description',
        'company_logo', 'company_website']);
    });
}
```

Don't run the migration just yet because I want to add a **user_id** field but I want to talk a little bit about relationships first and we'll do that in the next lesson.

Mass Assignment

We need to update the **JobListing** model to include the new fields. We also need to update the **\$fillable** property to include the new fields. Here is the updated model:

```
class JobListing extends Model
{
    use HasFactory;

    protected $fillable = [
        'title',
        'description',
        'salary',
        'tags',
        'job_type',
        'remote',
        'requirements',
        'benefits',
        'address',
        'city',
        'state',
        'zipcode',
        'contact_email',
        'contact_phone',
        'company_name',
        'company_description',
        'company_logo',
        'company_website'
    ];
}
```

```
    ];  
  }
```

In the next lesson, we will talk about relationships in Eloquent.